Institute for Computer Science VI, Autonomous Intelligent Systems, University of Bonn

Dr. N. Goerke
Friedrich-Hirzebruch-Allee 8, 53115 Bonn, Tel: +49 228 73-4167
E-Mail: goerke (at) ais.uni-bonn.de
http://www.ais.uni-bonn.de/WS/4204_L_NN.html

Exercises for module Technical Neural Networks (MA-INF 4204), WS26/27

Assignments Sheet 4, due: Tuesday 10.11.2026

3.11.2026

Assignment 16 (2 Points)

Describe and draw a typical learning curve for an MLP that is trained with Backpropagation of Error, single step learning strategy. Make sure, that the diagram is completely labeled.

Assignment 17 (2 Points)

Draw the graph of an (unrealistic) learning curve $f(z) = 2^{(2-z+4*sin(z))}$ with 4 different ways to scale the axes.
A: both axes scaled linear; B: one axis linear and one logarithmic, C: the other axis logarithmic and the first one linear, and D: both axes with logarithmic scaling.
What does the different scaling tell us with respect to a learning curve and the learning process?

Assignment 18 (2 Points)

Explain how an MLP can solve the encoder-decoder tasks: *8-3-8* and *8-2-8*.
Explain further why an MLP is not capable to solve the *8-1-8* task.

Either assignment 19 or assignment 20 has to be operated using AI help.
Report which AI-System helped you, which prompts you have decided to use. What the percentage did the AI-system contribute to your solution? In case you had to re-set your AI-session to get an answer, report why, when and how often. Have you discarded some of the AI-generated answers? If yes, Why?

Assignment 19 (3 Points) (AI help possible)

Derive a new learning rule ** for a Multi-Layer-Perceptron.
Start from the new objective function (cost function, error function) E^{**} and derive the new learning rule in analogy to Backpropagation of Error. The new cost function has been extended by a term that depends on all weights w_{ij}.
Write down all calculation steps, and give the formulas for calculating the δ^{**} in output- and hidden layer.

$$ {}^{p}E^{**} = \frac{1}{2}\sum_{m=1}^{M}({}^{p}\hat{y}_m \; - \; {}^{p}y_m)^2 + \beta\frac{1}{2}\sum_{i,j}(w_{ij})^2 $$

Assignment 20 (3 Points) (AI help possible)

Calculate, how to change the **input vector** ${}^p\mathbf{X}$ of a 3-layer, N-H-M MLP to decrease the single error ${}^pE$ using gradient descent.
All weights $w_{i,j}$ of the network are to remain constant, the transfer functions in hidden layer and output layer are the hyperbolic tangent.

The way how to change the input values can be determined in analogy to the derivation of backpropagation of error.
Derive the formulas for a 3-layer N-H-M MLP to calculate the partial derivative of the ${}^pE({}^px_n)$ with respect to components ${}^px_n$ of the N-dimensional input vector ${}^p\mathbf{X}$.

$$\frac{\partial\, {}^pE({}^px_n)}{\partial\, {}^px_n} = ?$$

Programming Assignment PA-B (5 Points, Due date: Wed 10.11.2026)

Implement and train an MLP to solve so called encoder-decoder tasks.
You can use your own MLP implementation or a working one from a library. In case you use a library, explicitly tell us which one and describe how to access it (e.g. a detailed description in the comments of your source code).

Start to implement an MLP to do the *8-3-8* task, with 3 layers, having 8 input neurons, 3 hidden neurons and 8 output neurons. The input and teacher values shall be 0.0 or 1.0. Choose appropriate transfer functions and learning parameters (report them).
Train the network with Backpropagation of Error, until a resonable result is achieved.

Print or depict the resulting input and output values after you finished learning. While learning, show the learning curve, and try to visualize the states of the three hidden neurons for all 8 patterns.

Redo the above steps for the *8-2-8* task.

Redo the task for either the *50-2-50* case or the *64-6-64* case.